

Module 8) Web Technologies in Java

1. Webpage with Menu, Form, Table, Images, and Lists

```
<!DOCTYPE html>
<html>
<head>
  <title>Lab Exercise 1</title>
</head>
<body>

<!-- Navigation Menu -->
<nav>
  <a href="#">Home</a> |
  <a href="#">About</a> |
  <a href="#">Contact</a>
</nav>

<!-- Form -->
<h3>User Form</h3>
<form>
  <label>Name:</label>
  <input type="text"><br><br>

  <label>Email:</label>
  <input type="email"><br><br>

  <input type="submit" value="Submit">
</form>

<!-- Table -->
<h3>User Data</h3>
<table border="1">
  <tr>
    <th>Name</th>
    <th>Email</th>
  </tr>
```

```

    <tr>
      <td>Rushi</td>
      <td>Rushi@email.com</td>
    </tr>
  </table>

  <!-- Image -->
  

  <!-- Lists -->
  <h3>Languages</h3>
  <ul>
    <li>HTML</li>
    <li>CSS</li>
  </ul>

  <ol>
    <li>Java</li>
    <li>Servlet</li>
  </ol>

</body>
</html>

```

2. Inline, Internal, and External CSS

```

<!DOCTYPE html>
<html>
  <head>
    <title>CSS Types</title>

    <!-- Internal CSS -->
    <style>
      h2 {

```

```
        color: blue;
    }
</style>
```

```
<!-- External CSS -->
<link rel="stylesheet" href="style.css">
</head>
<body>

<p style="color:red;">This is Inline CSS</p>

<h2>This is Internal CSS</h2>

<div class="box">This is External CSS</div>

</body>
</html>
```

```
style.css
.box {
    background-color: lightgreen;
    padding: 10px;
}
```

3. Margin and Padding Demonstration

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .box1 {
            margin: 30px;
            background-color: lightblue;
        }
        .box2 {
            padding: 30px;
            background-color: lightcoral;
        }
    </style>
</head>
<body>
```

```
    }
  </style>
</head>
<body>

<div class="box1">Margin Example</div>
<div class="box2">Padding Example</div>

</body>
</html>
```

4. Pseudo-Classes Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    a:hover {
      color: red;
    }
    input:focus {
      background-color: yellow;
    }
  </style>
</head>
<body>

<a href="#">Hover over me</a><br><br>
<input type="text" placeholder="Focus here">

</body>
</html>
```

5. ID and Class Usage

```
<!DOCTYPE html>
<html>
<head>
  <style>
```

```

        #title {
            color: blue;
        }
        .text {
            color: green;
        }
    </style>
</head>
<body>

<h1 id="title">Unique Heading</h1>

<p class="text">Paragraph 1</p>
<p class="text">Paragraph 2</p>

</body>
</html>

```

6. Client-Server Communication Diagram (Explanation)

Flow:

Client sends request (browser)

Server processes request

Server sends response

Client displays output

7. Servlet to Display Request Headers

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```

public class HeaderServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        out.println("<h3>Request Headers</h3>");
    }
}

```

```

        out.println(req.getHeaderNames());
        res.setHeader("MyHeader", "LabExample");
    }
}

```

8. J2EE Architecture Explanation

Layers:

Presentation Layer – JSP, Servlets

Business Logic Layer – Java Beans

Data Layer – Database

Explanation:

Each layer has a specific role, making applications scalable and secure.

9. Simple CGI Program Using Java

```
import java.io.*;
```

```

public class HelloCGI {
    public static void main(String[] args) throws Exception {
        System.out.println("Content-type:text/html\n");
        System.out.println("<h1>Hello CGI</h1>");
    }
}

```

10. Servlet with Parameters + Advantages over CGI

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```

public class UserServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws IOException {

        String name = req.getParameter("username");

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("Welcome " + name);
    }
}

```

```
}  
}
```

11. Servlet using GenericServlet and HttpServlet (Comparison)

```
import java.io.*;
```

GenericServlet Example

```
import javax.servlet.*;
```

```
public class GenericDemo extends GenericServlet {  
    public void service(ServletRequest req, ServletResponse res)  
        throws IOException {  
  
        PrintWriter out = res.getWriter();  
        out.println("This is GenericServlet");  
    }  
}
```

HttpServlet Example

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
public class HttpDemo extends HttpServlet {  
    public void doGet(HttpServletRequest req, HttpServletResponse res)  
        throws IOException {  
  
        PrintWriter out = res.getWriter();  
        out.println("This is HttpServlet");  
    }  
}
```

Comparison

GenericServlet → Protocol independent

HttpServlet → HTTP specific (GET, POST)

HttpServlet is more commonly used

12. HttpServlet Handling GET and POST

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```

public class MethodServlet extends HttpServlet {

    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
        res.getWriter().println("GET Request Received");
    }

    public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
        res.getWriter().println("POST Request Received");
    }
}

```

13. Servlet Life Cycle Methods

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LifeCycleServlet extends HttpServlet {

    public void init() {
        System.out.println("Servlet Initialized");
    }

    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
        res.getWriter().println("Service Method Called");
    }

    public void destroy() {
        System.out.println("Servlet Destroyed");
    }
}

```

14. Servlet Configuration using web.xml

Servlet Code


```

public class ConfigServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
        res.getWriter().println("Configured using web.xml");
    }
}

```

web.xml

```

<servlet>
    <servlet-name>config</servlet-name>
    <servlet-class>ConfigServlet</servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name>config</servlet-name>
    <url-pattern>/config</url-pattern>
</servlet-mapping>

```

15. ServletConfig Example

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

```

```

public class ConfigParamServlet extends HttpServlet {

    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {

        ServletConfig sc = getServletConfig();
        String name = sc.getInitParameter("admin");

        res.getWriter().println("Admin Name: " + name);
    }
}

```

web.xml

```

<init-param>
    <param-name>admin</param-name>
    <param-value>Rushi</param-value>

```

</init-param>

16. Login Form (JSP + Servlet + RequestDispatcher)

login.jsp

```
<form action="login" method="post">
  Username: <input type="text" name="user">
  <input type="submit">
</form>
```

LoginServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws IOException, ServletException {

        String user = req.getParameter("user");

        RequestDispatcher rd;
        if(user.equals("admin")) {
            rd = req.getRequestDispatcher("success.jsp");
        } else {
            rd = req.getRequestDispatcher("error.jsp");
        }
        rd.forward(req, res);
    }
}
```

17. ServletContext + Listener

ServletContext Example

```
ServletContext ctx = getServletContext();
ctx.setAttribute("college", "ABC College");
```

Listener

```
import javax.servlet.*;
```

```

public class AppListener implements ServletContextListener {

    public void contextInitialized(ServletContextEvent e) {
        System.out.println("Application Started");
    }

    public void contextDestroyed(ServletContextEvent e) {
        System.out.println("Application Stopped");
    }
}

```

18. Fetch Init Parameters using ServletConfig

```

public class InitParamServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {

        ServletConfig sc = getServletConfig();
        res.getWriter().println(sc.getInitParameter("db"));
    }
}

```

19. Registration Form (JSP → Servlet → JSP)

register.jsp

```

<form action="register" method="post">
    Name: <input type="text" name="name">
    <input type="submit">
</form>

```

RegisterServlet

```

public class RegisterServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws IOException, ServletException {

        req.setAttribute("name", req.getParameter("name"));
        RequestDispatcher rd = req.getRequestDispatcher("result.jsp");
        rd.forward(req, res);
    }
}

```

```
}  
}
```

20. Filter for Validation

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
public class ValidationFilter implements Filter {
```

```
    public void doFilter(ServletRequest req, ServletResponse res,  
        FilterChain chain)
```

```
        throws IOException, ServletException {
```

```
        String name = req.getParameter("name");
```

```
        if(name == null || name.isEmpty()) {  
            res.getWriter().println("Invalid Input");
```

```
        } else {  
            chain.doFilter(req, res);
```

```
        }
```

```
    }
```

```
}
```

web.xml

```
<filter>
```

```
    <filter-name>validate</filter-name>
```

```
    <filter-class>ValidationFilter</filter-class>
```

```
</filter>
```

```
<filter-mapping>
```

```
    <filter-name>validate</filter-name>
```

```
    <url-pattern>/*</url-pattern>
```

```
</filter-mapping>
```

21. Filter to Check Empty Form Fields

HTML Form (input.jsp)

```
<form action="submit" method="post">
```

```
    Name: <input type="text" name="name"><br><br>
```

```
Email: <input type="text" name="email"><br><br>
<input type="submit" value="Submit">
</form>
```

ValidationFilter.java

```
import java.io.*;
import javax.servlet.*;

public class EmptyCheckFilter implements Filter {

    public void doFilter(ServletRequest req, ServletResponse res,
        FilterChain chain)
        throws IOException, ServletException {

        String name = req.getParameter("name");
        String email = req.getParameter("email");

        if(name == null || name.isEmpty() || email == null || email.isEmpty()) {
            RequestDispatcher rd = req.getRequestDispatcher("input.jsp");
            rd.forward(req, res);
        } else {
            chain.doFilter(req, res);
        }
    }
}
```

web.xml

```
<filter>
    <filter-name>check</filter-name>
    <filter-class>EmptyCheckFilter</filter-class>
</filter>

<filter-mapping>
    <filter-name>check</filter-name>
    <url-pattern>/submit</url-pattern>
</filter-mapping>
```

If empty → returns to form
If not → request continues to servlet

22. JSP Using JSTL, Scriptlets, and Implicit Objects

jstlExample.jsp

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

```
<%  
    String[] subjects = {"Java", "HTML", "CSS"};  
    request.setAttribute("list", subjects);  
%>
```

```
<h3>Using JSTL (forEach)</h3>  
<ul>  
    <c:forEach var="s" items="${list}">  
        <li>${s}</li>  
    </c:forEach>  
</ul>
```

```
<h3>Using Scriptlet</h3>  
<%  
    out.println("Server Name: " + request.getServerName());  
%>
```

```
<h3>Implicit Objects</h3>  
<p>Context Path: <%= request.getContextPath() %></p>
```

23. Login System Using JSP, Servlet, Cookies, and Sessions

login.jsp

```
<form action="login" method="post">  
    Username: <input type="text" name="user"><br><br>  
    Password: <input type="password" name="pass"><br><br>  
    <input type="submit" value="Login">  
</form>
```

LoginServlet.java

```
import java.io.*;
```

```

import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {

    public void doPost(HttpServletRequest req, HttpServletResponse res)
        throws IOException, ServletException {

        String user = req.getParameter("user");
        String pass = req.getParameter("pass");

        if(user.equals("admin") && pass.equals("123")) {

            // Session
            HttpSession session = req.getSession();
            session.setAttribute("user", user);

            // Cookie
            Cookie ck = new Cookie("username", user);
            res.addCookie(ck);

            res.sendRedirect("welcome.jsp");
        } else {
            res.sendRedirect("login.jsp");
        }
    }
}

```

welcome.jsp

```

<%
    String user = (String) session.getAttribute("user");
    if(user == null) {
        response.sendRedirect("login.jsp");
    }
%>

<h2>Welcome <%= user %></h2>

```